

14-Linux-Git-Docker

Linux / Git / Docker 面试题

一、Linux

问：常用 Linux 命令？

答：

进程管理：

```
ps aux | grep java      # 查看进程
top / htop              # 实时监控
kill -9 <pid>           # 强制杀进程
jps -l                  # Java 进程
```

网络：

```
netstat -tlnp           # 查看端口占用
ss -tlnp                # 更快的 netstat
curl -I http://url      # 测试 HTTP
wget url                # 下载文件
ping host               # 测试连通性
tcpdump -i eth0 port 80 # 抓包
```

文件操作：

```
find / -name "*.log"    # 查找文件
tail -f app.log          # 实时看日志
grep "ERROR" app.log     # 搜索关键字
grep -r "keyword" ./    # 递归搜索
awk '{print $1}' file    # 列处理
sed 's/old/new/g' file  # 替换
```

磁盘：

```
df -h                  # 磁盘使用
du -sh *               # 目录大小
lsblk                  # 块设备
```

权限：

```
chmod 755 file          # 修改权限
chown user:group file    # 修改所有者
```

问：如何排查 CPU 100%?

答：

```
1. top                  # 找到高 CPU 的 PID
2. top -H -p <pid>      # 找到高 CPU 的线程 TID
3. printf "%x\n" <tid>   # TID 转 16 进制
4. jstack <pid> | grep <hex_tid> -A 30 # 查看线程栈
```

问：如何排查内存问题？

答：

```
free -h                # 系统内存
jmap -heap <pid>        # Java 堆信息
jmap -dump:format=b,file=heap.hprof <pid> # dump 堆
# 用 MAT 分析 heap dump
```

问：如何查看网络性能指标？

答：

```
ss -s                  # Socket 统计
netstat -an | awk '/^tcp/ {++S[$NF]} END {for(a in S) print a, S[a]}' # TCP 状态统计
iftop                  # 实时流量
iostat -x 1            # IO 统计
vmstat 1                # 系统整体状态
```

问：软链接和硬链接？

答：

```
ln file hard_link      # 硬链接 (同 inode)
ln -s file soft_link   # 软链接 (符号链接)
```

硬链接不能跨分区、不能链目录；软链接可以。

问：常见 Shell 脚本面试题？

答：

```
# 统计 PV/UV (从 Nginx 日志)
awk '{print $1}' access.log | sort | uniq -c | sort -rn # PV by IP
awk '{print $7}' access.log | sort -u | wc -l # UV (按 URL 去重)

# 查找大文件
find / -size +100M -type f 2>/dev/null

# 查看端口被谁占用
lsof -i :8080
```

二、Git

问：Git 工作流程？

答：

```
工作区 (Working Directory)
    git add
暂存区 (Staging Area / Index)
    git commit
本地仓库 (Local Repository)
    git push
远程仓库 (Remote Repository)
```

问：常用 Git 命令？

答：

```
git clone <url> # 克隆
git status # 状态
git add . # 暂存
git commit -m "msg" # 提交
git push origin main # 推送
git pull # 拉取 (fetch + merge)
git branch feature # 创建分支
git checkout feature # 切换分支
git merge feature # 合并分支
git rebase main # 变基
git log --oneline # 日志
git stash # 暂存修改
git cherry-pick <commit> # 挑选提交
git reset --hard HEAD~1 # 回退 (危险)
git revert <commit> # 安全回退 (新提交)
```

问：git merge 和 git rebase 的区别？

答：

对比	merge	rebase
历史	保留分支历史，有合并提交	线性历史，无合并提交
冲突	一次解决	可能多次解决
安全	不改已有提交	改写提交历史
场景	公共分支合并	个人分支整理

原则：已 push 的公共分支不要 rebase。

问：如何解决冲突？

答：

```
1. git pull / merge 提示冲突
2. 打开冲突文件，手动编辑（删除 <<<< ==== >>>> 标记）
3. git add <file>
4. git commit
```

问：git reset 和 git revert 的区别？

答：

对比	reset	revert
原理	移动 HEAD 指针	创建新提交反向操作
历史	删除提交历史	保留历史
安全	危险（已 push 不要用）	安全
场景	本地未 push 的提交	已 push 的提交

三、Docker

问：Docker 核心概念？

答：

概念	说明
镜像 (Image)	只读模板，类似类
容器 (Container)	镜像的运行实例，类似对象
仓库 (Registry)	镜像存储 (Docker Hub)
Dockerfile	构建镜像的脚本

问：常用 Docker 命令？

答：

```
docker build -t myapp:1.0 .      # 构建镜像
docker images                    # 列出镜像
docker run -d -p 8080:80 myapp   # 后台运行，端口映射
docker ps                       # 运行中容器
docker ps -a                    # 所有容器
docker stop <id>                # 停止
docker rm <id>                  # 删除容器
docker rmi <image>              # 删除镜像
docker logs -f <id>             # 查看日志
docker exec -it <id> /bin/bash   # 进入容器
docker-compose up -d            # 编排启动
```

问：Dockerfile 常用指令？

答：

```
FROM openjdk:17-jdk-slim      # 基础镜像
WORKDIR /app                  # 工作目录
COPY target/app.jar app.jar   # 复制文件
EXPOSE 8080                   # 暴露端口
ENV JAVA_OPTS=""              # 环境变量
RUN apt-get update            # 构建时执行
CMD ["java", "-jar", "app.jar"] # 启动命令（可被覆盖）
ENTRYPOINT ["java", "-jar"]    # 入口点（不可被覆盖）
```

CMD vs ENTRYPOINT：CMD 可被 `docker run` 参数覆盖；ENTRYPOINT 不可。

问：Docker 网络模式？

答：

模式	说明
bridge	默认，容器有独立 IP，通过 docker0 桥接
host	共享宿主机网络，无端口映射
none	无网络
container	共享另一个容器网络

问：Docker 数据卷？

答：

```
docker volume create mydata
docker run -v mydata:/app/data myapp # 命名卷
docker run -v /host/path:/container/path myapp # 绑定挂载
```

作用：数据持久化，容器删除后数据不丢失。

问：Docker 和虚拟机的区别？

答：

对比	Docker	虚拟机
隔离	进程级（namespace）	硬件级（Hypervisor）
启动	秒级	分钟级
资源	共享内核，轻量	独立 OS，重
性能	接近原生	有损耗

问：docker-compose 是什么？

答：多容器编排工具，通过 `docker-compose.yml` 定义服务、网络、卷，一条命令启动整个应用栈。

```
version: '3'
services:
  app:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - mysql
  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
```

四、Linux 排查深入

问：top 命令如何排查性能问题？

答：

```
top
# 关键行:
# load average: 1/5/15 分钟负载, > CPU 核数说明排队
# %Cpu(s): us 用户态 sy 内核态 wa IO等待 id 空闲
# PID USER %CPU %MEM COMMAND
```

交互命令： - P：按 CPU 排序 - M：按内存排序 - 1：显示每个 CPU 核心 - H：显示线程（配合 jstack 定位）

判断： - %wa 高 → IO 瓶颈（iostat、iotop） - %sy 高 → 系统调用多、上下文切换（vmstat、pidstat）
- %us 高 → 应用计算密集（jstack 看线程栈）

问：jstack 和 jmap 实战用法？

答：

jstack（线程栈，定位死锁/CPU 高）：

```
jstack -l <pid> > thread.dump
# 查找: deadlock、BLOCKED 线程、RUNNABLE 中耗 CPU 的线程
# CPU 高: top -H -p pid 得 TID → printf "%x\n" TID → jstack 中搜 nid=0x...
```

jmap（堆内存，定位 OOM）：

```
jmap -heap <pid> # 堆配置和使用概况
jmap -histo <pid> | head -20 # 对象实例数、占用排行
jmap -dump:format=b,file=heap.hprof <pid> # 完整堆转储
# MAT / JProfiler 分析: 大对象、泄漏引用链
```

频繁 Full GC：

```
jstat -gcutil <pid> 1000 # 每秒打印 GC 统计
# 关注 FGCT、GCT 增长趋势
```

问：Nginx 核心配置有哪些？

答：

```

# 全局
worker_processes auto;
events {
    worker_connections 10240;
    use epoll;
}

http {
    # 反向代理
    upstream backend {
        server 127.0.0.1:8080 weight=5;
        server 127.0.0.1:8081;
        keepalive 32;
    }

    server {
        listen 80;
        server_name example.com;

        # 限流: 10MB 共享内存, 每秒 10 请求
        limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
        limit_req zone=api burst=20 nodelay;

        location /api/ {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        # 静态资源
        location ~* \.(js|css|png)$ {
            expires 7d;
            gzip on;
        }
    }
}

```

常用: `rewrite` 重定向、`ssl_certificate` HTTPS、`access_log` 日志格式。

五、Docker 深入

问: docker-compose 多服务编排实战?

答:


```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: prod
      MYSQL_HOST: mysql
    depends_on:
      mysql:
        condition: service_healthy
    networks:
      - app-net
    deploy:
      replicas: 2
      resources:
        limits:
          cpus: '0.5'
          memory: 512M

  mysql:
    image: mysql:8
    volumes:
      - mysql-data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 3
    networks:
      - app-net

  redis:
    image: redis:7-alpine
    networks:
      - app-net

volumes:
  mysql-data:

networks:
  app-net:
    driver: bridge
```

命令: `docker-compose up -d`、`docker-compose logs -f app`、`docker-compose scale app=3`。

问: Docker 镜像优化有哪些实践?

答: 1. 多阶段构建: 编译阶段 + 运行阶段, 减小最终镜像 2. Alpine 基础镜像: 5MB vs Ubuntu 70MB 3.

合并 RUN 指令: 减少层数 4. .dockerignore: 排除无关文件 5. 非 root 用户运行: `USER appuser`

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:17-jre-alpine
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

六、Kubernetes 基础

问：K8s 核心概念？

答：

概念	说明
Pod	最小部署单元，1+ 容器共享网络/存储
Deployment	管理 Pod 副本、滚动更新
Service	稳定访问入口，ClusterIP/NodePort/LoadBalancer
Ingress	HTTP/HTTPS 路由，七层负载均衡
ConfigMap/Secret	配置和密钥
Namespace	资源隔离

架构：Master（API Server、Scheduler、Controller Manager、etcd） + Node（kubelet、kube-proxy、容器运行时）。

问：K8s 常用命令？

答：

```
kubectl get pods -n default           # 查看 Pod
kubectl describe pod <name>           # 详情和事件
kubectl logs -f <pod> -c <container> # 日志
kubectl exec -it <pod> -- /bin/sh     # 进入容器
kubectl apply -f deployment.yaml      # 部署
kubectl scale deployment app --replicas=3 # 扩缩容
kubectl rollout status deployment/app  # 滚动更新状态
kubectl rollout undo deployment/app    # 回滚
```

排查：kubectl describe 看 Events；CrashLoopBackOff 查 logs；Pending 查资源/调度。

问：K8s 和 Docker 的关系？

答：– Docker 是容器运行时（构建、运行容器） – K8s 是容器编排平台（调度、扩缩容、服务发现、自愈） – K8s 1.24+ 默认用 **containerd** 替代 Docker shim，但镜像仍可用 `docker build` 构建推送到 Registry

七、Git 深入

问：git rebase 冲突如何解决？

答：

```
git checkout feature
git rebase main
# 冲突时:
# 1. 编辑冲突文件, 解决 <<<< ==== >>>> 标记
git add <resolved-files>
git rebase --continue
# 若想放弃: git rebase --abort

# 多次冲突: 每解决一个 commit 的冲突就 continue
# 全部完成后: git push -f origin feature (仅个人分支!)
```

rebase 冲突 vs merge 冲突：– merge：一次解决所有冲突 – rebase：每个 commit 可能冲突，需逐个解决（历史更干净）

黄金法则：不要 rebase 已 push 到公共分支的提交。

问：git stash 和 cherry-pick 的使用场景？

答：

stash（临时储藏）：

```
git stash save "wip: feature X"
git checkout hotfix-branch
# 修完回来
git checkout feature
git stash pop
```

cherry-pick（拣选提交）：

```
git cherry-pick <commit-hash>    # 将某提交应用到当前分支
git cherry-pick A..B              # 拣选 A 之后到 B 的提交 (不含 A)
```

场景：热修复从 main cherry-pick 到 release 分支；只要某次 fix 不要整个 merge。

问：.gitignore 和 git clean?

答：

```
# .gitignore
target/
*.class
.idea/
.env
```

```
git clean -fd # 删除未跟踪的文件和目录 (危险, 先 git clean -n 预览)
git rm --cached file # 停止跟踪已提交的文件
```

图解加深

专题	链接	要点
Linux 命令	Linux命令	进程、网络、磁盘排查
Git 命令	Git命令	工作流、分支策略
Docker	Docker面试题	镜像、网络、数据卷
性能排查	结合 OS 图解	top → jstack/jmap → MAT

大厂追问

1. Load Average 三个数字什么意思？ 1/5/15 分钟内平均活跃进程数（运行 + 等待 CPU），> CPU 核数表示过载。
2. jstack 看到大量 BLOCKED 怎么办？ 查锁竞争，搜 waiting to lock；配合 jstack -l 看持有锁的线程。
3. Nginx 502 和 504 区别？ 502 上游返回无效响应（服务挂了）；504 上游超时（upstream timed out）。
4. docker-compose 和 K8s 怎么选？ 开发/小规模用 compose；生产集群、自动扩缩容、服务发现用 K8s。
5. git rebase 和 merge 在团队中的规范？ feature 分支 rebase main 保持线性；合并到 main 用 merge 或 squash merge。
6. 容器 OOM 怎么排查？ docker stats 看内存；dmesg | grep oom；调整 -m 限制或 JVM -Xmx。